

**Method:**

For this problem I used disjoint set union to come up with an  $O(n^2)$  overall algorithm.

**Approach:**

While thinking about this problem I realized that the final answer would involve finding connected components and then further research led me to realize that DSU was a good approach because the problem essentially becomes, find all of the components and then you can remove all but one stone from each component, leaving the final result to become (number of stones - number of components). The algorithm starts by creating singleton sets for each stone, then comparing each pair of stones to see if they share a row or column, and if they do combining them.

**Reasoning:**

This algorithm is correct because DSU will find all of the components and then it can simply subtract the number of components from the total number of stones (this is correct because within each component you can remove all but one stone), which leaves you with only one stone from each component not removed.

**Reflection:**

I used these test cases:

Input: stones = `[[0,0],[0,1],[1,0],[1,2],[2,1],[2,2]]`

Output: 5

Input: stones = `[[0,0],[0,2],[1,1],[2,0],[2,2]]`

Output: 3

Input: stones = `[[0,0]]`

Output: 0

**Time Complexity:**

$O(n^2)$